DESPLIEGUE DE APLICACIONES WEB — CURSO 2026/2027

Observabilidad Logs, métricas y alertas

Tema 3 — Publicación y ejecución de aplicaciones web



Objetivo de la sesión

Punto de partida

Un servicio puede responder y seguir siendo difícil de administrar.

Necesitamos describir qué falla, cuándo y a quién afecta.

Qué trabajaremos

Logs estructurados y centralizados.

Tráfico, errores y latencia.

Paneles y alertas que ayudan a actuar.

La observabilidad permite comprender el sistema a partir de sus señales.

Una incidencia, tres señales

La misma lentitud se investiga desde perspectivas complementarias

Caso: una petición a **/api/productos** tarda **1,84 s**

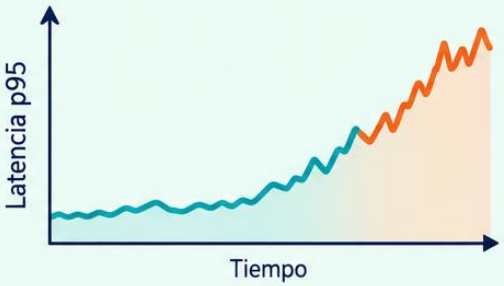
LOGS
¿Qué ocurrió?

```
ruta=/api/productos
estado=200
duracion_s=1.84
```

Un evento concreto con contexto

Permite localizar esta petición lenta.

MÉTRICAS
¿Cómo evoluciona?



Tendencia ilustrativa de muchas peticiones

Permite detectar si la lentitud se repite.

TRAZAS
¿Dónde se consume el tiempo?



Recorrido temporal de una petición

En este ejemplo, la consulta ocupa gran parte del tiempo.

Las señales se complementan: evento, tendencia y recorrido.

Las trazas requieren instrumentación. En esta sesión trabajaremos principalmente con logs y sus agregaciones.

Alcance de la observación

Datos de peticiones

Podemos contar respuestas y calcular latencias.

Un 200 también puede ser lento.

Una petición aislada no describe todo el servicio.

Otras señales

CPU, memoria y colas requieren mediciones específicas.

Las trazas necesitan instrumentación y contexto.

Los spans anidados no se suman sin más.

La práctica se centra en logs y agregaciones obtenidas a partir de ellos.

Access log y error log

Una petición, dos registros complementarios

El access log muestra el resultado; el error log ayuda a explicar el fallo



ACCESS LOG

¿Qué recibió el cliente?

```
{
   "ruta": "/api/productos",
   "estado": 502,
   "duracion_s": 0.012,
   "destino": "172.20.0.5:8080"
}
```

Campos con nombre para filtrar y agrupar.

ERROR LOG

¿Qué problema detectó Nginx?

Fallo al conectar con el upstream: conexión rechazada

```
Destino: 172.20.0.5:8080
Ruta: /api/productos
```

Mensaje explicativo simplificado.

Del registro a una consulta

Filtrar: `estado = 502` | **Contar:** errores por ruta | **Agrupar:** destinos afectados

El resultado y el detalle técnico aportan contexto para investigar.

Campos ilustrativos: dependen del log_format. Para correlacionar, añade fecha, hora e identificador de petición.

Qué registra Nginx

Access log

Normalmente registra al finalizar la petición.

Permite consultar ruta, estado y otros campos elegidos.

Puede haber filtros o registro desactivado.

Error log

Recoge problemas detectados por Nginx.

No hay una entrada obligatoria por cada respuesta de error.

El detalle de un 500 puede estar en el log de la aplicación.

Los campos del access log dependen del formato configurado.

Duración observada por el proxy

```
$request_time = 0.042 s = 42 ms
```

También influyen la recepción de la petición y el envío al cliente.

No representa toda la experiencia del navegador.

Nginx mide desde los primeros bytes leídos del cliente hasta el registro tras enviar la respuesta.

La duración del proxy no equivale exclusivamente a ejecución del backend.

Campos con nombre y unidades

```
{
  "fecha": "2026-10-15T10:00:00+00:00",
  "metodo": "GET",
  "ruta": "/api/productos",
  "estado": 200,
  "duracion_s": 0.042,
  "destino": "172.20.0.5:8080"
}
```

Unidades explícitas y contexto permiten filtrar, agrupar y comparar.

La herramienta de consulta debe interpretar correctamente los tipos numéricos.

Destinos, reintentos y contexto

Destinos del proxy

$upstream_addr registra los backends contactados.

Puede contener varias direcciones si hubo reintentos.

$upstream_status ayuda a interpretar los resultados.

Contexto de la petición

Fecha con zona horaria.

Servicio e instancia.

Identificador de petición cuando se propaga y registra.

Las IP de contenedores pueden cambiar.

Una dirección registrada puede corresponder a un intento fallido.

Un formato JSON para el access log

```
log_format observabilidad escape=json
    '{"fecha":"$time_iso8601",'
    '"metodo":"$request_method",'
    '"ruta":"$uri",'
    '"estado":$status,'
    '"duracion_s":$request_time,'
    '"destino":"$upstream_addr"}';

access_log /dev/stdout observabilidad;
```

Cada evento ocupa una línea. El error log conserva su configuración independiente.

log_format se declara en el contexto http de Nginx.

Datos útiles y datos sensibles

Para poder consultar

Conservar unidades y tipos.

Escapar los caracteres especiales.

Tratar valores ausentes y destinos múltiples.

Para reducir exposición

Excluir contraseñas y tokens.

Evitar cookies de sesión y datos personales innecesarios.

Revisar rutas y parámetros de URL.

Los registros también necesitan permisos y acceso controlado.

Centralización de registros



Patrón conceptual: la recogida de stdout y stderr depende de la configuración de la plataforma.

Primera consulta desde Compose

```
docker compose logs --tail=100 -f
```

La centralización añade búsqueda, conservación y agregaciones.

La recogida depende de la configuración de la plataforma.

Un fichero dentro del contenedor no aparece automáticamente en stdout o stderr.

Compose puede reunir la salida de varios servicios del proyecto.

Las cuatro señales de oro y RED

Qué observar en un servicio web

Cuatro señales de oro para entender el comportamiento del servicio

TRÁFICO



¿Cuántas peticiones llegan?

Peticiones por segundo

RED · Rate

ERRORES



¿Cuántas peticiones fallan?

Número y porcentaje de fallos

RED · Errors

LATENCIA



¿Cuánto tardan las peticiones?

Distribución de tiempos y p95

RED · Duration

SATURACIÓN



¿Cuánto se acerca el recurso a su límite?

Colas, conexiones y capacidad disponible

Señales de infraestructura

Access logs: tráfico, errores HTTP y latencia registrada por el proxy.

Saturación: requiere normalmente mediciones específicas de recursos.

RED ayuda a observar servicios. La saturación completa la visión de sus recursos.

Los campos y el periodo de observación deben estar definidos. Un estado HTTP no detecta todos los fallos de negocio.

Interpretación de los indicadores

Peticiones y fallos

Rate: peticiones por unidad de tiempo.

Errores: recuento y porcentaje.

Hay que definir qué es un fallo. Un 200 puede ocultar un fallo de negocio.

Recursos y cobertura

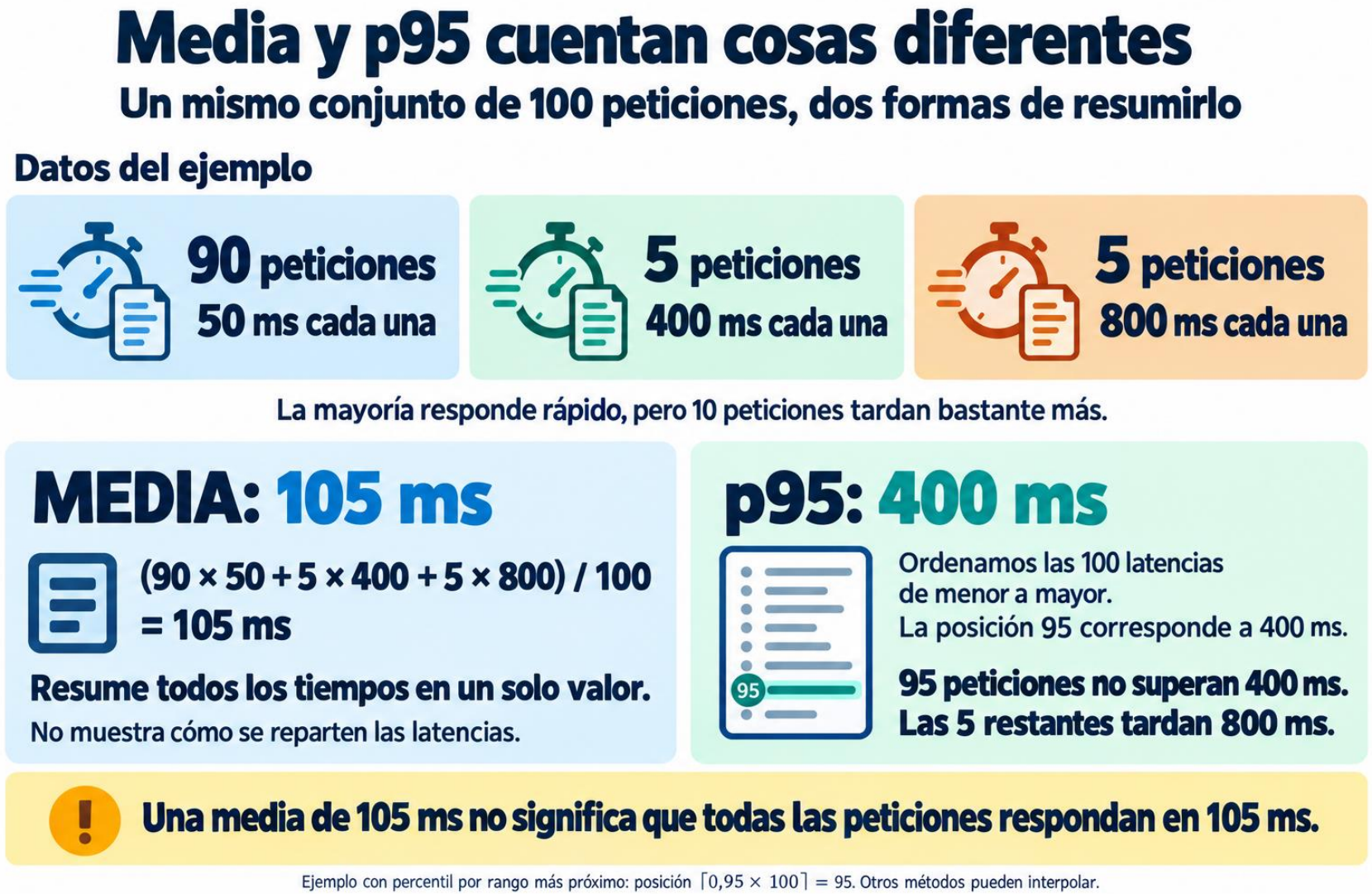
Una cola puede indicar falta de capacidad.

Utilización y saturación son distintas.

Filtros o pérdidas de logs limitan lo que podemos concluir.

El indicador describe el conjunto que se ha recogido y seleccionado.

Media y percentil 95



Cómo comparar latencias

Mismo conjunto de comparación

Ruta o servicio definidos.

Mismo intervalo y unidades.

Número de peticiones suficiente.

Separar éxitos y fallos si ayuda.

Interpretación del p95

Aproximadamente el 95 % no supera ese tiempo.

No es el máximo ni la media de las más lentas.

El método puede interpolar o aproximar.

No se obtiene el p95 global haciendo la media de los p95 de las réplicas.

Un panel para responder preguntas

Estado del servicio

¿Cambió la demanda?
Peticiones por segundo.

¿Dónde se concentra el fallo?
Recuento y porcentaje por ruta.

Investigación

¿Qué rutas son lentas?
Media y p95 con tamaño de muestra.

¿Qué destinos reciben tráfico?
Peticiones o intentos por destino.

Desde un gráfico debemos poder volver a los registros relevantes.

De un panel a una alerta accionable

Pregunta del panel: ¿están aumentando los errores del servicio?

De un panel a una alerta accionable

Una alerta útil relaciona una condición con una persona y una actuación

1 · Evaluar



Condición de ejemplo
Más de 10 respuestas 5xx en los últimos 2 minutos

Ventana móvil de datos

2 · Confirmar



Persistencia opcional
La condición sigue cumpliéndose durante 2 minutos

Tiempo de espera antes del aviso

3 · Avisar y actuar



Destinatario
Responsable del servicio

Acción
Revisar rutas afectadas, aplicación y dependencias

Ventana y persistencia son distintas: qué datos miro y cuánto espero.

Pico aislado



Si la condición deja de cumplirse antes del tiempo de espera, no se envía este aviso.

Problema sostenido



Si la condición se mantiene, el aviso pide una actuación concreta.

El umbral debe tener sentido para el tráfico y la gravedad del servicio.

Ejemplo didáctico: revisa también el porcentaje de errores y el volumen de peticiones. Evita avisos repetidos sin acción.

Ventana de datos y persistencia

Ventana de 2 minutos

A las 10:04 se cuentan los 5xx de los últimos 2 minutos.

Si hay más de 10, la condición se cumple y comienza la espera.

Persistencia de 2 minutos

Para avisar a partir de las 10:06, debe seguir cumpliéndose en las evaluaciones.

Si deja de cumplirse, no se envía este aviso.

Cada evaluación utiliza una nueva ventana móvil de datos.

Alertas con contexto y actuación

Volumen y gravedad

10 fallos entre 20 peticiones no equivalen a 10 entre 100 000.

Con poco tráfico, el porcentaje también puede oscilar mucho.

El umbral del ejemplo es didáctico.

Respuesta al aviso

Asignar responsable y acción.

Agrupar avisos relacionados.

Definir cuándo se considera recuperado.

Evitar repeticiones sin actuación.

Una ausencia de datos puede ser un fallo de recogida.

Retención, acceso y coste

Conservación

Decidir qué guardar y durante cuánto tiempo.

Distinguir rotación local y retención central.

Vigilar el espacio disponible.

Coste y utilidad

El detalle y los índices consumen recursos.

Limitar el acceso a los registros.

Revisar qué aporta cada dato y qué se pierde al filtrarlo.

La observación consume recursos y necesita mantenimiento.

Cierre de la sesión

Qué debemos poder explicar y comprobar

Qué señal aporta contexto para cada pregunta.

Cómo estructurar, recoger y consultar los logs.

Qué representan tráfico, errores, duración y saturación.

Cómo interpretar media y p95 sobre un mismo conjunto.

Qué condición merece un aviso y quién debe actuar.

La siguiente sesión mirará dentro del backend y abordará su estado y rendimiento.